DDA5001 Homework 3

The Chinese University of Hong Kong, Shenzhen

Problem 1(a): Regularization and five-fold validation

For a fit on m observations, let $X \in \mathbb{R}^{m\times 9}$ have columns $1, x, \ldots, x^8$. The objective, including a penalty on the intercept, is

$$J_\lambda(\theta) = \frac{1}{m}\|X\theta - y\|_2^2 + \lambda\|\theta\|_2^2.$$

Differentiating and setting the gradient to zero gives

$$\nabla J_\lambda = \frac{2}{m}X^\top(X\theta - y) + 2\lambda\theta = 0, \qquad (X^\top X + m\lambda I_9)\widehat{\theta}_\lambda = X^\top y.$$

Thus

$$\boxed{\widehat{\theta}_\lambda = (X^\top X + m\lambda I_9)^{-1}X^\top y.}$$

For any nonzero v, $v^\top(X^\top X + m\lambda I)v = \|Xv\|^2 + m\lambda\|v\|^2 > 0$ when $\lambda > 0$. The solution is unique. The code evaluates this equation with `np.linalg.solve`, without explicitly forming an inverse; $m = 10$ for the final fit.

The original CSV row IDs define validation folds

$$V_1 = \{6, 0\},\ V_2 = \{3, 9\},\ V_3 = \{2, 1\},\ V_4 = \{4, 8\},\ V_5 = \{5, 7\}.$$

For each of the 16 prescribed candidates, the supplied fitting routine trains on the eight remaining rows. The completed scoring function computes

$$E_f(\lambda) = \frac{1}{2}\sum_{i\in V_f}\left(f_{\widehat{\theta}_{\lambda,-f}}(x_i) - y_i\right)^2, \qquad \mathrm{CV}(\lambda) = \frac{1}{5}\sum_{f=1}^{5}E_f(\lambda).$$

Rows are never sorted or renumbered. Taking the first minimum over ascending candidates implements the exact-tie rule. The selected value is

$$\boxed{\lambda^\star = 0.01, \qquad \mathrm{CV}(\lambda^\star) = 0.04826737.}$$

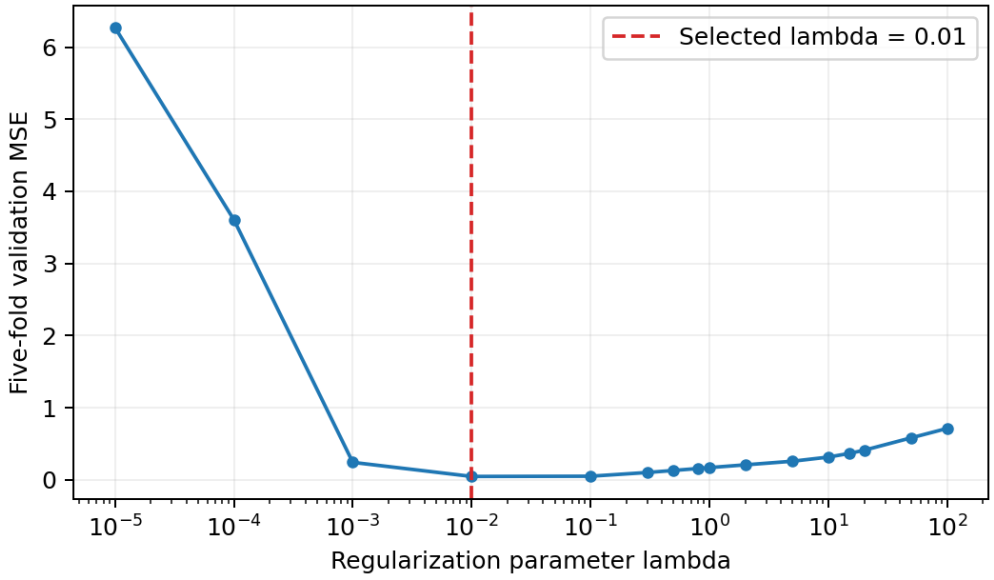


Figure 1: Five-fold mean validation MSE, with a logarithmic λ axis.

Problem 1(b): Refit and final evaluation

After selecting $\lambda^\star$ using only training-set folds, the regularized model is refitted on all ten training rows. The baseline uses the supplied unregularized least-squares solver. Both models are then evaluated on the ten independent test observations.

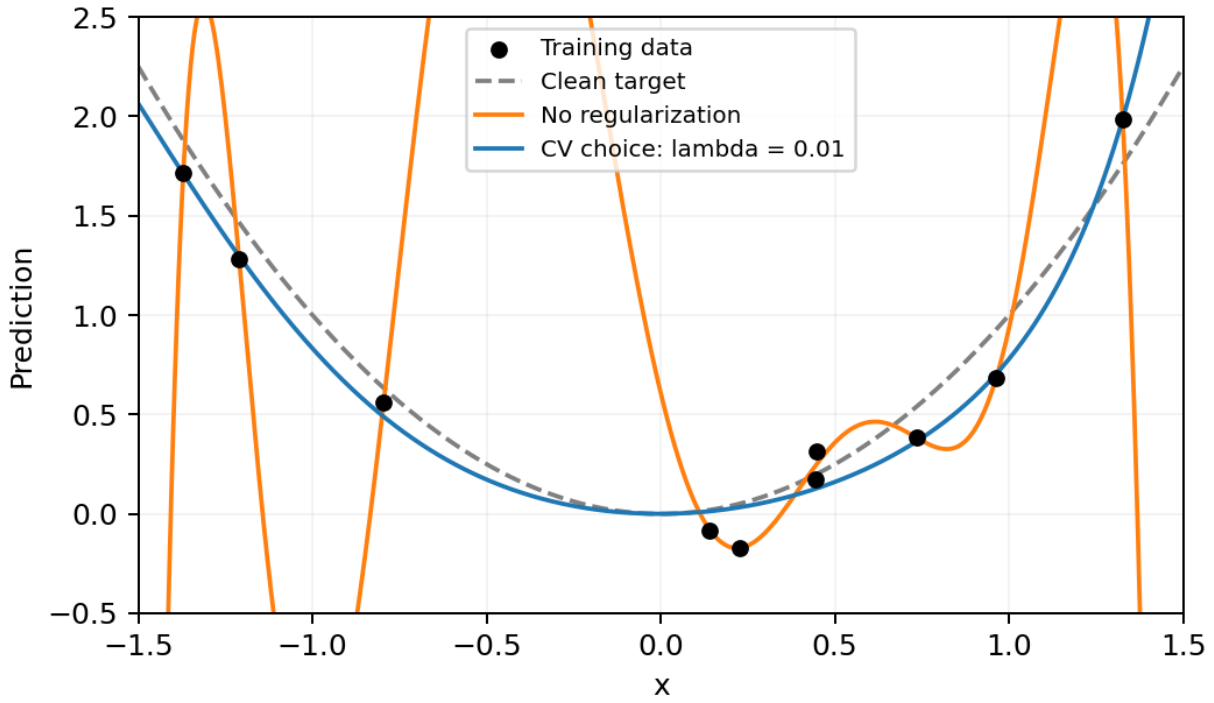


Figure 2: Degree-8 fits, the original training points and the clean target x^2. The prescribed display limits are $x \in [-1.5, 1.5]$, $y \in [-0.5, 2.5]$; parts of the unregularized curve extend beyond this window.

Fit	Training MSE	Test MSE
Unregularized ($\lambda = 0$)	0.00090436	2.08003393
Validation-selected ($\lambda = 0.01$)	0.00933397	0.04021308

The unregularized fit closely matches the noisy training observations but oscillates strongly between them, whereas regularization increases training MSE and substantially reduces test MSE in this run. The selected curve follows the clean quadratic trend more closely, illustrating a useful reduction in overfitting rather than a guarantee that every regularizer improves prediction. Validation chooses λ without using test outcomes, so the test set remains a separate final assessment instead of becoming another tuning set.

Implementation and checks. Only `fit_ridge_final` and `score_and_select_cv` in `p1/p1.py` were completed. A small analytic check with $X = I_2$, $y = (2, 4)^\top$ and $\lambda = 0.5$ returns $(1, 2)^\top$, checking the factor m and the penalty on every coordinate. A synthetic exact tie selects the smaller candidate. These checks supplement, rather than replace, the supplied-data experiment.

Data and outputs. The two CSVs are unchanged from the teacher package. Running `p1.py` produces the complete candidate-by-fold table in `cross_validation.csv`, a `summary.json` with coefficients and errors, and the figures shown here. These numerical files are generated outputs; none of the reported errors uses the clean curve in place of the noisy observed targets.

Problem 2(a): Centered PCA and eigenfaces

The 400 original 112×92 faces are ordered by subject $1, \ldots, 40$, then image $1, \ldots, 10$. The supplied loader divides pixels by 255; the completed matrix constructor flattens each image in row-major order without further normalization:

$$X = [x_1, \ldots, x_{400}] \in \mathbb{R}^{10304 \times 400}, \quad \mu = \frac{1}{400} X\mathbf{1}, \quad X_c = X - \mu\mathbf{1}^\top.$$

The code uses the thin SVD

$$X_c = U\Sigma V^\top, \qquad U \in \mathbb{R}^{10304 \times 400},$$

with singular values in descending order. The first k columns of U form an orthonormal basis A_k minimizing the rank-k reconstruction error. No 10304×10304 covariance matrix is formed.

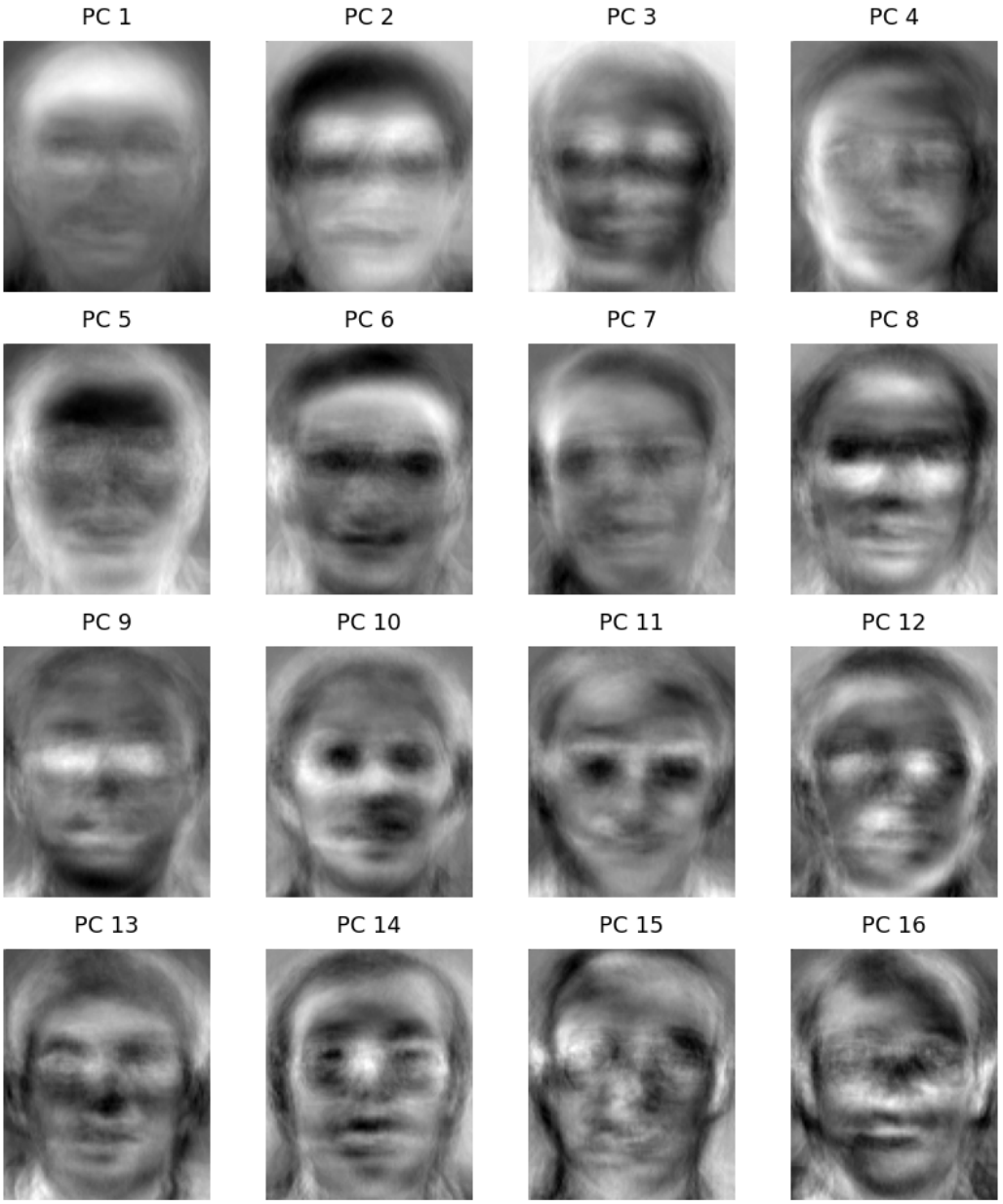


Figure 3: First 16 eigenfaces in descending singular-value order, read left to right and top to bottom. Display signs follow the supplied convention; reversing a basis vector's sign does not change its subspace or reconstruction.

Problem 2(b): Coordinates and reconstruction

For each k, the completed function returns

$$\Theta_k = A_k^\top (X - \mu \mathbf{1}^\top) \in \mathbb{R}^{k \times 400}, \qquad \widehat{X}_k = \mu \mathbf{1}^\top + A_k \Theta_k.$$



Figure 4: Image 1 of subjects 1–5, in row order (columns $0, 10, 20, 30, 40$ of X). Each row shows the original, $k = 20$ reconstruction and $k = 100$ reconstruction.

Components k	Whole-dataset MSE $\|\widehat{X}_k - X\|_F^2/(10304 \cdot 400)$
20	0.00734732
100	0.00271577

Increasing k from 20 to 100 preserves more detail around the eyes, hair and glasses and lowers the measured reconstruction error. The nested PCA subspaces retain more variation, but require five times as many coordinates per face. MSE is computed on all pixels of all 400 unclipped reconstructions; only the displayed images are clipped to $[0, 1]$.

Problem 3(a): Prediction before running

The following prediction was recorded in the AI-assisted workflow before model checks or training. Two affine layers without an activation collapse to

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

Therefore B remains affine in x. Its effective matrix has rank at most 64, but every 10×784 matrix has rank at most 10; width 64 is sufficient to represent every matrix available to A. More explicitly, choose the first ten rows of W_1 to be any desired W, set its other rows to zero, and take $W_2 = [I_{10}\ 0]$, $b_1 = 0$, $b_2 = b$.

A and B consequently have the same input-to-logit function class, although their parameterization and optimization trajectories may differ. ReLU allows C to use different affine pieces for different activation patterns, adding nonlinear representational capacity that may help with digits. Comparing B and C keeps width, depth, biases and parameter counts fixed, isolating the architectural addition of ReLU under the prescribed training protocol; no exact accuracy or universal ranking was predicted.

Problem 3(b): Model construction

The only changes to `p3/p3.py` are the three model constructors:

Model	Layers, in forward order	Output	Parameters
A	Linear$(784, 10)$	10 logits	7,850
B	Linear$(784, 64)$; Linear$(64, 10)$	10 logits	50,890
C	Linear$(784, 64)$; ReLU; Linear$(64, 10)$	10 logits	50,890

Every linear layer includes a bias. An input batch is $N \times 784$; B/C produce an intermediate $N \times 64$ tensor, then $N \times 10$ logits. C applies $\max(z, 0)$ elementwise only after the first affine layer. There is no output ReLU or Softmax: the supplied cross-entropy function consumes raw scores. The counts are $784 \cdot 10 + 10$ for A and $(784 \cdot 64 + 64) + (64 \cdot 10 + 10)$ for B/C; ReLU has no trainable parameters.

Checks. The supplied `--check-models` command returned finite outputs of shape $(2, 10)$ with the counts above. Additional automated checks inspected the actual layer types and biases and confirmed that B agrees with its collapsed affine formula in float64 at tolerance 10^{-12}.

Fixed experiment. The supplied split has 6,000 training, 2,000 validation and 2,000 test images, class-balanced with seed 666. The unchanged framework trains for 30 epochs using minibatches of 128, SGD with momentum 0.9, learning rate 0.1, no weight decay, and training seed 666. It uses one CPU thread and deterministic PyTorch algorithms. Epoch 0 is an untrained plotted baseline; eligible checkpoints are epochs 1–30. The checkpoint and model choice use validation cross-entropy only, before test evaluation.

Problem 3(c): Results and interpretation

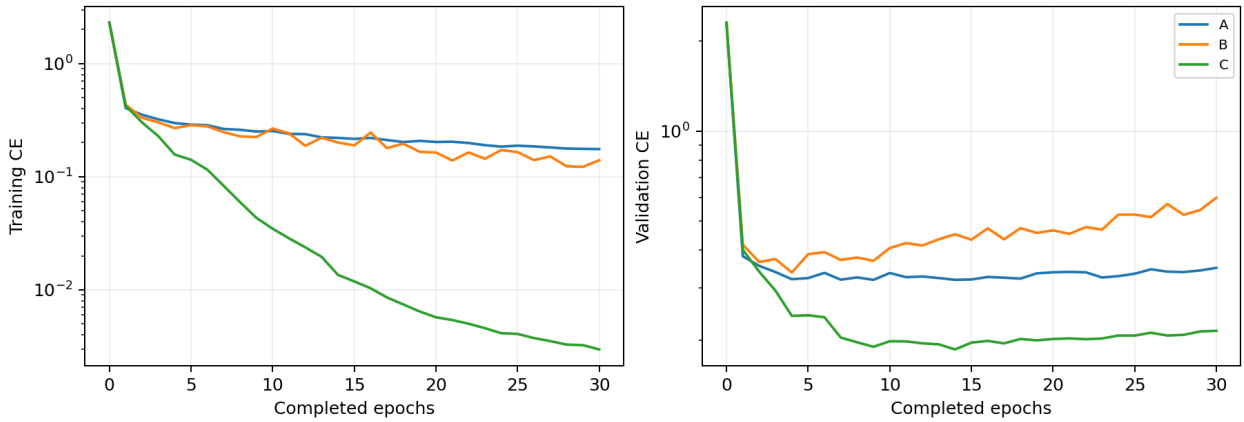


Figure 5: Training and validation cross-entropy (CE) for A, B and C. The supplied plotting code uses logarithmic vertical axes.

Model	Selected epoch	Validation CE	Test CE	Test accuracy
A	9	0.318305	0.277242	91.45%
B	4	0.336629	0.290094	90.80%
C	14	0.186219	0.170503	95.20%

Validation choice. C is preferred because its selected checkpoint has the smallest validation CE, 0.186219. The reported test values are evaluations of each model's validation-selected state; test accuracy was not the selection criterion.

A versus B. B has slightly worse selected validation and test CE than A in this run, despite having more parameters and lower final training CE. This is consistent with the prediction that stacking affine layers does not enlarge the input-to-logit function class: factorizing the weights changes how optimization proceeds, so equivalent function classes need not yield equal trained solutions.

B versus C. With the same 50,890 parameters and identical layer dimensions, C lowers selected validation CE from 0.336629 to 0.186219 and raises test accuracy from 90.80% to 95.20%. This supports the predicted usefulness of the hidden nonlinearity in this fixed experiment. It does not prove that ReLU always wins across data splits or seeds, and an A–C difference alone would mix activation and parameter-count effects.

Training–validation gaps and checkpoint selection.

	Selected epoch		Epoch 30	
Model	Training CE	Validation CE	Training CE	Validation CE
A	0.251042	0.318305	0.175604	0.349079
B	0.268865	0.336629	0.139791	0.598827
C	0.013566	0.186219	0.002974	0.214923

All three models end with lower training CE but higher validation CE than at their selected epochs. B's validation curve rises most strongly, while C's training CE approaches zero without a matching late validation improvement. This widening gap is evidence of overfitting in this run; restoring the earlier validation minimum avoids simply taking a later, better-fitting training state. Validation selection is still an estimate from one finite split, not a guarantee of the best unknown population or test loss.

Problem 3(d): Actual coding-agent collaboration

Tool and scope. OpenAI Codex was used in the desktop app for model construction, numerical checks, experiment execution and report drafting. The assistant identifies itself as GPT-6; no more specific model version is asserted here. The case below describes this actual interaction, rather than a fabricated student debugging exchange.

Actual user prompt. The initial Chinese request, translated into English, was:

> "Please help me complete hw03; you may refer to the completion format of previous assignments and AGENTS.md."

After the correct assignment files were supplied, the user asked, "Now take another look." The task then included the three model-construction functions required by Problem 3(b).

Relevant agent reply. Before running the experiment, Codex stated the following construction plan (translated from the original Chinese response):

> "The concrete implementations are `Linear(784,10)`, `Linear(784,64)` $\rightarrow$ `Linear(64,10)`, and `Linear(784,64)` $\rightarrow$ `ReLU` $\rightarrow$ `Linear(64,10)`; all outputs remain raw logits."

It also recorded that two layers without an activation remain affine, whereas hidden ReLU enables nonlinear piecewise-affine maps. The three actual return statements in the submitted `p3/p3.py` implement these definitions using `nn.Linear` and `nn.Sequential`.

How the proposal was checked. The checks were executed by the assistant, not presented as checks personally performed by the student. The provided framework reported:

Model	Trainable parameters	Output shape
A	7,850	$(2, 10)$
B	50,890	$(2, 10)$
C	50,890	$(2, 10)$

The additional model tests passed: they checked the exact sequence of layer types, all biases, output dimensions and B's equality to its collapsed affine map within 10^{-12} in float64. Together with inspection of the constructors, these checks rule out an unintended output Softmax, missing bias or misplaced activation.

Decision and evidence. The proposed constructors were retained because the definitions, counts and checks agree with the three required formulas. No model-construction bug or artificial repair is claimed. The finite outputs and completed fixed-framework run provide execution evidence; they do not replace the algebra explaining A/B equivalence. The original interaction is retained in this Codex chat, and the local pre-run note and check logs preserve the timing and evidence.

Authorship disclosure. The mathematical explanations and this report are AI-assisted drafts. The numerical results come from actual local execution of the submitted implementation. The student remains responsible for checking understanding and expressing the derivations and explanations in their own words before submission; this record does not claim that personal step has already occurred.